

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнила:

Смирнова Анна

Группа J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию.

Ход работы

1. Проектирование модульной архитектуры приложения

Весь программный код файла `script.js` был разделен на независимые логические блоки:

- `api.js` — модуль для сетевого взаимодействия и работы с API
- `render.js` — модуль отрисовки элементов интерфейса и визуализации данных
- `modals.js` — модуль управления модальными окнами
- `utils.js` — набор вспомогательных функций для обработки данных и дат
- `main.js` — центральный модуль, инициализирующий приложение и обрабатывающий события

2. Настройка серверной среды и проектирование API

Была развернута серверная среда разработки на базе `json-server` с поддержкой аутентификации (`json-server-auth`). Спроектирована архитектура данных, включающая независимые сущности: пользователи, транзакции, цели, категории, банковские аккаунты и правила импорта. Настроен механизм авторизации, позволяющий безопасно разграничивать данные между пользователями

3. Реализация слоя доступа к данным

С нуля был разработан слой работы с API, использующий современный асинхронный стандарт `fetch` с `async/await`. Создана универсальная функция-обертка `apiFetch`, обеспечивающая:

- централизованную обработку HTTP-запросов и ответов
- автоматическую передачу токена в заголовках авторизации
- обработку ошибок сервера и управление состоянием сессии пользователя

4. Разработка механизмов интеграции и бизнес-логики

- Реализован функционал подключения и отключения банковских аккаунтов с сохранением состояния в базе данных.
- Создан алгоритм автоматической обработки транзакций, использующий пользовательские правила (сопоставление ключевых слов в описании с целевыми категориями).
- Реализован механизм симуляции импорта данных с сервера, который в реальном времени применяет правила категоризации к новым транзакциям, обеспечивая высокую степень автоматизации учета

5. Разработка системы визуализации и аналитики

Для наглядного представления финансового состояния пользователя была интегрирована библиотека Chart.js с плагином chartjs-plugin-datalabels

- Реализованы функции динамической отрисовки графиков, привязанные к асинхронным данным: «Динамика баланса» (линейный график) и «Расходы по категориям» (круговая диаграмма)
- Настроена логика автоматического пересчета графиков при изменении временных интервалов и обновлении данных в базе, что обеспечивает актуальность аналитики в реальном времени

6. Интерактивность и интерфейс

Реализована динамическая отрисовка всех компонентов системы: списков транзакций, прогресс-баров целей и таблиц категорий. Приложение спроектировано таким образом, что любое изменение данных на сервере мгновенно отражается во всех элементах интерфейса без необходимости перезагрузки страницы.

Вывод

В ходе выполнения лабораторной работы проект был переведен на клиент-серверную архитектуру. Реализовано взаимодействие с REST API, обеспечена авторизация и изоляция данных пользователей. Проведен рефакторинг кода, разделивший логику на независимые модули (api.js, render.js и др.), что повысило удобство поддержки приложения. Внедрена система аналитики на базе Chart.js и функционал автоматизации импорта транзакций с использованием пользовательских правил. Результатом стало масштабируемое веб-приложение, использующее современные стандарты асинхронного взаимодействия, полностью готовое к дальнейшей эксплуатации и расширению.